

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**



BÁO CÁO THỰC TẬP CƠ SỞ

**ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG GỢI Ý
ANIME HYBRID RECOMMENDER**

ỨNG DỤNG HỌC MÁY VÀ DEEP LEARNING

Giảng viên hướng dẫn : ThS. Nguyễn Hoàng Anh
Sinh viên thực hiện : Đào Trung Hiếu
MSV : B23DCCN298

Hà Nội, 6/2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành đến Học viện Công nghệ Bưu chính Viễn thông và Khoa Công nghệ Thông tin 1 đã tạo điều kiện và đưa học phần Thực tập cơ sở vào chương trình giảng dạy, giúp em có cơ hội cọ xát với thực tế.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến thầy Nguyễn Hoàng Anh đã trực tiếp hướng dẫn và định hướng cho em trong suốt thời gian thực hiện đề tài. Những góp ý chuyên môn của thầy, đặc biệt trong việc ứng dụng Deep Learning vào hệ thống, đã giúp em giải quyết được nhiều vướng mắc và hoàn thiện sản phẩm. Trong quá trình làm việc cùng thầy, em đã học hỏi được tinh thần làm việc nghiêm túc, khoa học và cách vận dụng kiến thức lý thuyết vào việc mô hình hóa, triển khai thực tiễn. Điều này không chỉ nâng cao kỹ năng lập trình mà còn giúp em tự tin hơn cho các dự án sau này.

Do giới hạn về thời gian nghiên cứu và kinh nghiệm thực tiễn, báo cáo cũng như sản phẩm chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự đánh giá và góp ý từ thầy để đề tài được hoàn thiện hơn, đồng thời rút ra những bài học quý giá cho bản thân.

Kính chúc thầy luôn dồi dào sức khỏe và thành công trong sự nghiệp giảng dạy. Mong rằng trong tương lai, em sẽ tiếp tục được học hỏi thầy nhiều hơn nữa!

Em xin chân thành cảm ơn!

MỤC LỤC

I. PHẦN MỞ ĐẦU	4
1. Đặt vấn đề	4
2. Mục tiêu đề tài	4
3. Phạm vi nghiên cứu	4
II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	5
1. Tổng quan về Hệ thống khuyến nghị (Recommender Systems)	5
2. Phương pháp Lọc cộng tác (Collaborative Filtering) và SVD	6
3. Phương pháp Lọc theo nội dung & Xử lý ngôn ngữ tự nhiên	7
3.1 Mô hình cổ điển: TF-IDF	7
3.2 Mô hình Học sâu: Sentence-BERT	7
3.3 Độ tương đồng Cosine (Cosine Similarity)	7
3.4 Thước đo đánh giá mô hình (RMSE)	8
3.5 Công nghệ và kiến trúc phần mềm	8
III. PHÂN TÍCH VÀ TIỀN XỬ LÝ DỮ LIỆU	9
1. Mô tả tập dữ liệu (Dataset)	9
2. Tiền xử lý dữ liệu	9
2.1 Lọc nhiễu và giảm độ thừa thớt	10
2.2 Kỹ thuật Metadata Soup	10
3. Trích xuất đặc trưng (Feature Engineering)	10
IV. KIẾN TRÚC VÀ XÂY DỰNG MÔ HÌNH HYBRID	11
1. Sơ đồ kiến trúc tổng thể	11
1.1. Lớp Giao diện (Frontend - ReactJS & Vite)	11
1.2. Lớp Máy chủ (Backend - Python Flask)	11
1.3. Lớp Trí tuệ nhân tạo (AI Models & Processing)	12
2. Huấn luyện mô hình	12
2.1 Cấu hình siêu tham số cho mô hình SVD	12
2.2 Nhúng vector văn bản bằng Sentence-BERT	12
3. Thuật toán Hybrid	12
V. TRIỂN KHAI ỨNG DỤNG VÀ ĐÁNH GIÁ	13
1. Đánh giá độ chính xác	13
2. Triển khai giao diện	15
2.1. Chức năng Đồng bộ MyAnimeList (MAL Sync)	15
2.2. Chức năng Nhập liệu thủ công (Manual Import)	17
2.3. Chức năng AI Engine Toggle	18
VI. KẾT LUẬN & TÀI LIỆU THAM KHẢO	18
1. Kết luận	18
2. Tài liệu tham khảo	19

I. PHẦN MỞ ĐẦU

1. Đặt vấn đề

- Trong kỷ nguyên công nghệ số hiện nay, ngành công nghiệp giải trí nói chung và phim hoạt hình Nhật Bản (Anime) nói riêng đang chứng kiến sự bùng nổ mạnh mẽ về mặt số lượng lẫn thể loại. Trên các nền tảng cơ sở dữ liệu lớn như MyAnimeList, số lượng tác phẩm đã lên tới hàng chục ngàn bộ phim. Mặc dù sự đa dạng này mang lại nhiều lựa chọn, nhưng nó đồng thời tạo ra hiện tượng "quá tải thông tin" đối với người dùng.
- Khi đứng trước một thư viện nội dung khổng lồ, người dùng gặp rất nhiều khó khăn trong việc tìm kiếm những bộ phim thực sự phù hợp với "gu" cá nhân. Hiện nay, phần lớn các hệ thống tìm kiếm đều dựa vào các bộ lọc thể loại truyền thống (Action, Romance, Isekai...). Tuy nhiên, phương pháp này đã trở nên bão hòa và bộc lộ nhiều hạn chế, bởi lẽ hai bộ phim cùng thể loại có thể mang lại trải nghiệm cảm xúc hoàn toàn trái ngược nhau. Do đó, việc xây dựng một Recommender System cá nhân hóa, có khả năng hiểu được hành vi người dùng và phân tích sâu vào ngữ nghĩa của từng bộ phim là một bài toán mang tính cấp thiết và có giá trị thực tiễn cao.

2. Mục tiêu đề tài

- **Về mặt thuật toán:** Nghiên cứu và xây dựng một hệ thống Hybrid Recommender. Hệ thống này là sự kết hợp giữa thuật toán Lọc cộng tác dự đoán điểm số thông qua Phân rã ma trận (SVD - Singular Value Decomposition) và thuật toán Lọc theo nội dung ứng dụng mạng nơ-ron Học sâu (Deep Learning) để phân tích ngữ nghĩa ngôn ngữ tự nhiên (Sentence-BERT) kết hợp TF-IDF.
- **Về mặt ứng dụng:** Phát triển một ứng dụng Web hoàn chỉnh (Web Application) với giao diện hiện đại, thân thiện, cho phép người dùng tương tác trực tiếp với AI. Hệ thống hỗ trợ tính năng đồng bộ hóa lịch sử xem phim tự động từ MyAnimeList, hoặc cho phép người dùng tự tìm kiếm và chấm điểm thủ công để huấn luyện AI đưa ra các gợi ý theo thời gian thực.

3. Phạm vi nghiên cứu

- **Về dữ liệu:** Nghiên cứu được thực hiện trên tập dữ liệu Anime công khai từ Kaggle và MyAnimeList, bao gồm thông tin của hơn 24.000 bộ phim và hàng triệu lượt đánh giá đã qua các bước tiền xử lý và làm sạch. Số liệu được thống kê đến 2023.

<https://www.kaggle.com/datasets/dbdmobile/myanimelist-dataset/data>

<https://myanimelist.net/>

- **Về phương pháp:** Tập trung chuyên sâu vào hai hướng tiếp cận:
 - Lọc cộng tác dựa trên người dùng Collaborative Filtering thông qua thuật toán SVD để giảm chiều dữ liệu

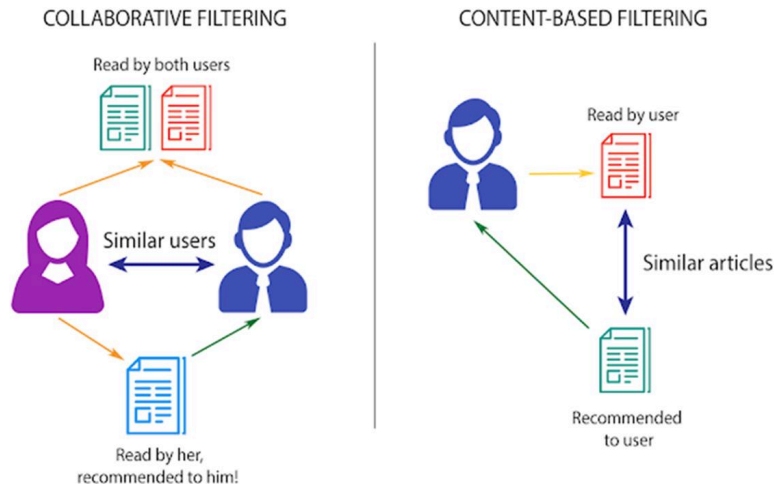
- Lọc theo nội dung dựa trên văn bản Content Filtering gồm phương pháp đếm từ khóa TF-IDF so sánh với trích xuất đặc trưng ngữ nghĩa sâu (Deep Text Embeddings).

Đề tài chưa đi sâu vào các hệ thống gợi ý theo thời gian thực dựa trên luồng nhấp chuột hay hệ thống phân tán quy mô lớn.

II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

1. Tổng quan về Hệ thống khuyến nghị (Recommender Systems)

- Hệ thống khuyến nghị hay còn gọi là hệ thống gợi ý (Recommender Systems – RS) là một lớp các hệ thống lọc thông tin có nhiệm vụ dự đoán "đánh giá" hoặc "sở thích" mà người dùng sẽ dành cho một vật phẩm vào sở thích/thói quen của họ trong quá khứ. Trong bối cảnh bùng nổ thông tin, hệ tư vấn giúp cá nhân hóa trải nghiệm người dùng, giữ chân khách hàng và tối ưu hóa việc phân phối nội dung.
- Ví dụ:
 - Shopee gợi ý các món hàng dựa trên lịch sử mua hàng của người dùng.
 - Netflix tự động gợi ý phim cho người dùng.
 - Facebook hiển thị quảng cáo những sản phẩm có liên quan đến từ khóa bạn vừa tìm kiếm; gợi ý kết bạn
- Dựa trên nguyên lý hoạt động, hệ tư vấn thường được chia thành 3 hướng tiếp cận chính:
 - Lọc cộng tác (Collaborative Filtering - CF): Dựa trên lịch sử tương tác giữa người dùng và vật phẩm như điểm đánh giá, lịch sử xem... để tìm ra những người dùng có sở thích tương đồng, từ đó đưa ra gợi ý.
 - Lọc theo nội dung (Content-Based Filtering - CBF): Dựa trên các đặc trưng của vật phẩm như thể loại, đạo diễn, tóm tắt nội dung... để gợi ý các vật phẩm tương tự với những gì người dùng đã thích trong quá khứ.
 - Hệ tư vấn lai ghép (Hybrid Recommender Systems): Kết hợp nhiều phương pháp (như CF và CBF) nhằm khắc phục nhược điểm của từng phương pháp đơn lẻ và tăng cường độ chính xác tổng thể.



2. Phương pháp Lọc cộng tác (Collaborative Filtering) và SVD

Phương pháp lọc cộng tác không yêu cầu kiến thức về bản thân các vật phẩm (không cần biết anime đó thuộc thể loại gì), mà chỉ tập trung khai thác Utility Matrix chứa các điểm đánh giá của người dùng cho vật phẩm.

Ví dụ cho Utility Matrix (với A, B, C, D, E, F là 6 user):

	A	B	C	D	E	F
Mưa nửa đêm	5	5	0	0	1	?
Cổ úa	5	?	?	0	?	?
Vùng lá me bay	?	4	1	?	?	1
Con cò bé bé	1	1	4	4	4	?
Em yêu trường em	1	0	5	?	?	?

Cấu trúc ma trận: Các hàng đại diện cho người dùng, các cột đại diện cho các vật phẩm. Giá trị tại mỗi ô là điểm số (từ 1 đến 5) mà người dùng đó đã chấm cho vật phẩm.

Vấn đề dữ liệu thưa: Trong thực tế, một người dùng chỉ xem vài chục phim trong số hàng chục ngàn phim. Do đó, ma trận tiện ích chứa tới 99% là các ô trống.

Trong các kỹ thuật CF, Mô hình nhân tố ẩn (Latent Factor Models), điển hình là thuật toán Phân rã ma trận (Matrix Factorization - SVD), được chứng minh là mang lại độ chính xác cao nhất, đặc biệt nổi tiếng sau cuộc thi Netflix Prize.

Thuật toán SVD (Singular Value Decomposition)

Để điền những ô trống trong ma trận tiện ích (tức là dự đoán điểm người dùng sẽ chấm cho phim chưa xem), hệ thống áp dụng thuật toán SVD.

- Thay vì dùng các công thức toán học phức tạp để tính toán trực tiếp, SVD sẽ "ép" ma trận tiện ích khổng lồ ban đầu vỡ ra thành 2 ma trận nhỏ hơn: Ma trận đặc trưng của Người dùng và Ma trận đặc trưng của Phim.
- Trong dự án, SVD được sử dụng từ thư viện `scikit-surprise`. Khi gọi lệnh `svd_model.fit(trainset)`, thuật toán sẽ tự động tìm ra các đặc trưng ẩn (latent features) để khớp với dữ liệu thực tế. Khi cần dự đoán điểm, chỉ cần gọi hàm `svd_model.predict(uid, iid)` là hệ thống sẽ trả về một điểm số ước tính cực kỳ sát với sở thích người dùng.

3. Phương pháp Lọc theo nội dung & Xử lý ngôn ngữ tự nhiên

Do SVD bị điểm yếu "Khởi động lạnh" (không thể gợi ý phim mới chưa ai xem), hệ thống bổ sung phương pháp Lọc theo nội dung (content-based) bằng cách đọc hiểu tóm tắt phim và thể loại.

3.1. Mô hình cổ điển: TF-IDF

Trong triển khai thực tế bằng thư viện `scikit-learn`, TF-IDF biến mỗi đoạn tóm tắt phim thành một mảng dữ liệu. Nếu từ khóa "sword" xuất hiện nhiều ở phim A và phim B, nó sẽ đánh dấu hai phim này giống nhau. Tuy nhiên, giới hạn thực chiến của TF-IDF là nó khá "máy móc", không phân biệt được từ đồng nghĩa (Ví dụ: "sword" và "blade").

3.2. Mô hình Học sâu: Sentence-BERT

Để giải quyết bài toán ngữ nghĩa, dự án tích hợp thư viện `sentence-transformers`. Sentence-BERT là một kiến trúc mạng nơ-ron Học sâu, thay vì đếm từ, nó đọc hiểu toàn bộ câu văn.

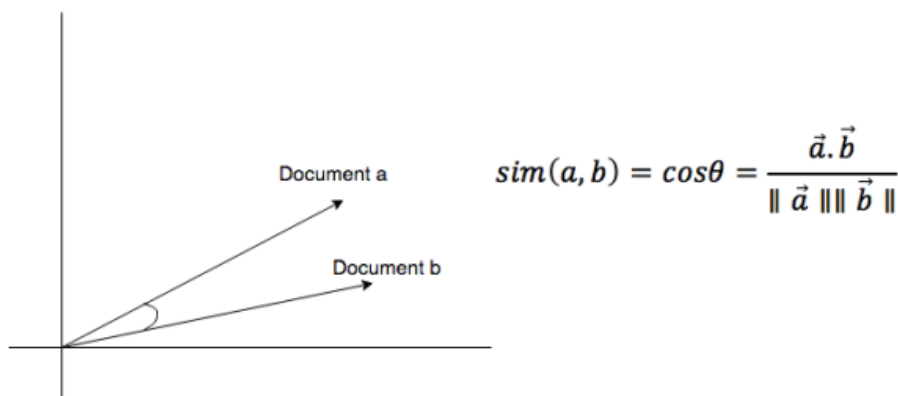
Khi đưa dữ liệu qua hàm `bert_model.encode()`, mỗi bộ phim sẽ được mã hóa thành một Vector đa chiều (Embeddings). Các bộ phim có chung độ tương đồng về thể loại hay nội dung/cốt truyện sẽ tự động hội tụ lại gần nhau trong không gian vector.

3.3. Độ tương đồng Cosine (Cosine Similarity)

Sau khi biến các bộ phim thành các vector toán học, hệ thống đo lường khoảng cách giữa chúng để tìm ra các phim giống nhau nhất bằng công thức Cosine Similarity:

$$sim(a, b) = \cos\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

Độ giống nhau của hai vector là 1 số trong đoạn $[-1, 1]$. Giá trị bằng 1 thể hiện hai vector hoàn toàn giống nhau. Giá trị bằng - 1 thể hiện hai vector này hoàn toàn trái ngược nhau.



Tuy nhiên khi dùng TF-IDF, vì tần suất đếm từ khóa không bao giờ là số âm nên giá trị cosine similarity trả về từ 0 đến 1. Điểm càng gần 1, hai bộ phim càng giống nhau về nội dung.

Cấu trúc ma trận Cosine này được xuất ra thành file .pkl để Backend tải trực tiếp lên RAM, giúp tốc độ tìm kiếm phim tương tự KNN (K-Nearest Neighbors) diễn ra chỉ trong vài mili-giây.

3.4 Thước đo đánh giá mô hình (RMSE)

Để đo lường xem hệ thống AI dự đoán chuẩn đến đâu, dự án sử dụng thước đo Căn bậc hai sai số toàn phương trung bình (RMSE). Cách thước đo này hoạt động trong thực tế:

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (Predicted_i - Actual_i)^2}{N}}$$

1. Máy dự đoán phim A được 8 điểm, nhưng người dùng thực tế chỉ chấm 6 điểm -> Sai số là 2.
2. Lấy sai số bình phương lên ($2^2 = 4$) để loại bỏ dấu âm và phạt nặng những dự đoán lệch quá xa.
3. Tính trung bình tất cả các sai số của hàng ngàn dự đoán, sau đó lấy căn bậc hai để đưa về lại thang điểm [1-10].

Trong quá trình huấn luyện, chỉ số này được gọi trực tiếp thông qua hàm `accuracy.rmse(predictions)`. Chỉ số RMSE càng nhỏ, chứng tỏ hệ thống dự đoán càng sát với thực tế.

3.5 Công nghệ và kiến trúc phần mềm

Để đưa các thuật toán AI trên thành một ứng dụng hoàn chỉnh, dự án kết hợp các công nghệ sau:

- **Xử lý dữ liệu & AI (Python):** Sử dụng Pandas để xử lý hàng triệu dòng dữ liệu, dọn dẹp các giá trị rỗng và trích xuất đặc trưng trước khi đưa vào mô hình học máy.
- **Máy chủ Backend (Flask RESTful API):** Khung ứng dụng Web nhẹ của Python. Flask đóng vai trò làm cầu nối, nhận yêu cầu chứa danh sách phim từ giao diện, nạp các file AI model, tính toán điểm lai ghép và trả kết quả Top 10 về dưới dạng JSON.
- **Giao diện Frontend (ReactJS & Vite):** ReactJS xử lý mượt mà các thao tác cập nhật dữ liệu liên tục như thanh trượt chấm điểm, tự động tìm kiếm, và hiển thị danh sách thẻ phim thông qua việc gọi API Axios.

III. PHÂN TÍCH VÀ TIỀN XỬ LÝ DỮ LIỆU

1. Mô tả tập dữ liệu (Dataset)

Đề tài sử dụng bộ dữ liệu Anime công khai được thu thập từ nền tảng MyAnimeList. Để tối ưu hóa tài nguyên RAM trong quá trình huấn luyện nhưng vẫn đảm bảo tính đa dạng của dữ liệu, hệ thống trích xuất khoảng 2.000.000 lượt đánh giá để đưa vào mô hình. Dữ liệu được chia làm 2 bảng chính:

Bảng thông tin Anime (anime-filtered.csv): Bảng này chứa metadata của các bộ phim, đóng vai trò là nguồn thông tin cốt lõi cho thuật toán lọc theo nội dung. Các trường quan trọng bao gồm:

- `anime_id`: Mã định danh duy nhất của bộ phim.
- `name`: Tên bộ phim.
- `genre`: Danh sách các thể loại phim, ví dụ: Action, Adventure, Fantasy....
- `synopsis`: Đoạn văn tóm tắt cốt truyện.
- `score`: Điểm số trung bình do cộng đồng đánh giá với thang 1-10.
- `members`: Tổng số người dùng đã thêm bộ phim này vào danh sách của họ.
- `dropped`: Số lượng người dùng đã xem nhưng bỏ ngang (không xem hết).

Bảng lịch sử đánh giá (user-filtered.csv): Bảng này lưu trữ ma trận tương tác giữa người dùng và phim. Các trường dữ liệu bao gồm:

- `user_id`: Mã định danh người dùng.
- `anime_id`: Mã phim mà người dùng tương tác.
- `rating`: Điểm số người dùng chấm cho bộ phim đó với thang 1-10.

2. Tiền xử lý dữ liệu:

Dữ liệu thô từ thực tế luôn chứa nhiều nhiễu và các giá trị không hợp lệ. Quá trình tiền xử lý được thực hiện qua các bước nghiêm ngặt sau:

2.1 Lọc nhiễu và giảm độ thưa thớt

- Loại bỏ điểm 0: Trên MyAnimeList, khi người dùng lưu một bộ phim vào danh sách "Plan to watch" (Dự định xem) nhưng chưa xem, hệ thống sẽ ghi nhận mức điểm là 0. Việc giữ lại các giá trị 0 này sẽ làm sai lệch nghiêm trọng thuật toán SVD (AI sẽ hiểu lầm là người dùng cực kỳ ghét bộ phim đó). Do vậy, toàn bộ các dòng có `rating = 0` đều bị loại bỏ.
- Lọc dữ liệu thừa: Để mô hình AI có thể tìm ra được quy luật đáng tin cậy, hệ thống áp dụng ngưỡng lọc:
 - Chỉ giữ lại những người dùng đã đánh giá lớn hơn 50 bộ phim giúp loại bỏ các "tài khoản rác".
 - Chỉ giữ lại những bộ Anime nhận được lớn hơn 50 lượt đánh giá giúp tăng độ tin cậy thống kê cho tập huấn luyện.
 - Kết quả : từ 2.000.000 rating xuống chỉ còn 1.019.961 rating sạch

2.2 Kỹ thuật Metadata Soup

- Metadata Soup: Gôm thể loại và tóm tắt thành một khối văn bản để AI đọc hiểu. Cột `genre` và `synopsis` (tóm tắt) được nối lại thành một chuỗi văn bản duy nhất.

```
anime_df['metadata_soup'] = anime_df['genre'] + " " +
anime_df['genre'] + " " + anime_df['synopsis']
```

Ở đây `genre` được lặp 2 lần để tăng cường hệ số so với tóm tắt. Bước này giúp các vector embeddings được sinh ra sau này mang đậm đặc trưng thể loại hơn.

3. Trích xuất đặc trưng (Feature Engineering):

Trong thực tế, một bộ phim có nội dung tương đồng với sở thích của người dùng chưa chắc đã là một bộ phim đáng xem (có thể cốt truyện giống nhưng chất lượng hình ảnh, đạo diễn rất tệ). Để khắc phục điểm mù này của thuật toán Content-Based, hệ thống đã nghiên cứu và phát triển một chỉ số tự tinh chỉnh gọi là Trọng số Chất lượng.

Trọng số này được tính toán dựa trên dữ liệu thống kê của cộng đồng, hoạt động như một hệ số nhân để "thưởng" cho các tác phẩm xuất sắc và "phạt" các tác phẩm có chất lượng kém.

Xây dựng công thức

Bước 1: Tính tỷ lệ drop của phim (người xem được vài tập thì bỏ ngang k xem nữa => phim tệ)

$\text{Drop_Rate} = \text{Dropped} / \text{Members}$

Bước 2: Chuẩn hóa điểm số trung bình: Sử dụng hàm `MinMaxScaler` để đưa điểm Score (thang 1-10) về khoảng [0, 1] nhằm đồng bộ đơn vị tính toán.

Bước 3: Tổng hợp Trọng số chất lượng:

$\text{Quality_Weight} = 1.0 + (\text{Normalized_Score} * 0.2) - (\text{Drop_Rate} * 1.5)$

Giải thích ý nghĩa công thức:

- **Mặc định 1.0:** Trọng số mặc định cho mọi bộ phim. Nếu Weight = 1.0, điểm gợi ý được giữ nguyên.
- **Thưởng điểm +0.2:** Một bộ phim có điểm số cao tuyệt đối có thể nhận được tối đa 20% điểm cộng thêm vào thuật toán gốc.
- **Phạt điểm -1.5:** Tỷ lệ bỏ ngang phim phản ánh sự thất vọng của người dùng. Trọng số phạt được thiết lập rất cao (1.5) để đánh tụt hạng những bộ phim bị cộng đồng cho là tệ.
- **Kiểm soát ranh giới:** Để tránh việc hệ số nhân trở thành số âm, hệ thống sử dụng hàm clip(lower=0.5) để đảm bảo trọng số của một bộ phim dù tệ đến đâu cũng không bao giờ rớt xuống dưới mức 0.5.

IV. KIẾN TRÚC VÀ XÂY DỰNG MÔ HÌNH HYBRID

1 Sơ đồ kiến trúc tổng thể:

Hệ thống được thiết kế theo mô hình Client-Server thông qua chuẩn giao tiếp RESTful API, đảm bảo tính rời rạc và dễ dàng mở rộng. Luồng xử lý dữ liệu được chia thành 3 lớp chính:

1.1. Lớp Giao diện (Frontend - ReactJS & Vite):

- Chịu trách nhiệm tiếp nhận tương tác từ người dùng (nhập Username MyAnimeList, tìm kiếm phim, chấm điểm thủ công).
- Quản lý trạng thái của ứng dụng, vd: ghi nhận người dùng đang bật công tắc AI nào (Deep Learning hay Classic).
- Đóng gói dữ liệu đầu vào thành định dạng JSON và gửi yêu cầu (HTTP POST) xuống máy chủ Backend thông qua thư viện Axios.

1.2. Lớp Máy chủ (Backend - Python Flask):

- Đóng vai trò là controller. Nhận luồng JSON từ frontend, tiến hành phân giải và kiểm tra tính hợp lệ của dữ liệu.
- Nếu người dùng chọn tính năng "MyAnimeList Sync", backend sẽ kích hoạt module cào dữ liệu để kéo danh sách phim lịch sử của người dùng đó về.
- Tải các model AI đã được huấn luyện sẵn lên RAM để phục vụ tính toán.
- Sau khi tính toán ra Top 10 bộ phim phù hợp nhất, hệ thống sử dụng cơ chế đa luồng để cào ảnh poster của 10 bộ phim này trong thời gian thực.
- Trả kết quả cuối cùng về cho Frontend hiển thị.

1.3. Lớp Trí tuệ nhân tạo (AI Models & Processing):

- Nhận danh sách phim lịch sử từ Backend. Đưa qua luồng xử lý Hybrid giữa SVD và BERT/TF-IDF.

2. Huấn luyện mô hình:

2.1. Cấu hình siêu tham số cho mô hình SVD

Bảng tinh chỉnh tham số:

Kịch bản	Tốc độ học (lr_all)	Điều chuẩn (reg_all)	Thời gian train	RMSE (SVD Độc lập)
#1	0.002	0.02	26s	1,2301
#2	0.005	0.02	25s	1,2805
#3 (chọn)	0.005	0.10	24s	1,2092
#4	0.010	0.20	25s	1,2597

Hệ thống sử dụng thư viện Scikit-Surprise và thiết lập các siêu tham số kiểm soát chặt chẽ quá trình huấn luyện:

- **n_epochs = 30:** Số vòng lặp huấn luyện. Mô hình sẽ duyệt qua tập dữ liệu 50 lần để tinh chỉnh các nhân tố ẩn .
- **lr_all = 0.005 (Learning Rate):** Tốc độ học. Con số này được đặt ở mức đủ nhỏ để thuật toán Gradient Descent từ từ tìm đến điểm hội tụ (sai số thấp nhất) mà không bị "trượt" qua mất; nhưng cũng không quá nhỏ để tránh việc huấn luyện diễn ra quá lâu.
- **reg_all = 0.1 (Regularization):** Hệ số điều chuẩn. Đây là chốt chặn quan trọng nhất để chống lại hiện tượng "Học vẹt" (Overfitting). Nó sẽ trừng phạt (giảm giá trị) các trọng số toán học quá lớn, ép AI phải đưa ra các mức điểm dự đoán bám sát với thực tế thay vì đoán các con số vô lý.

2.2. Nhúng vector văn bản bằng Sentence-BERT

Đối với nhánh xử lý ngôn ngữ tự nhiên, hệ thống sử dụng mô hình pre-trained **all-MiniLM-L6-v2** từ thư viện **sentence-transformers**. Đây là mô hình được Google tối ưu hóa, cực kỳ nhẹ nhưng giữ được độ chính xác rất cao.

- Quá trình huấn luyện diễn ra bằng cách đưa cột Metadata Soup qua hàm **encode()**.
- AI sẽ đọc hiểu từng đoạn của các bộ phim và nhả ra một ma trận nhúng. Mỗi bộ anime lúc này biến thành một vector tọa độ 384 chiều.
- Cuối cùng, hệ thống tính toán sẵn hàm Cosine Similarity cho toàn bộ 24.000 bộ phim, tạo thành một lưới liên kết nội dung và xuất ra file .pkl.

3. Thuật toán Hybrid:

Bước 1: Lấy điểm nền từ SVD

Với mỗi bộ phim ứng viên, hệ thống yêu cầu mô hình SVD dự đoán một mức điểm (Ví dụ: `svd_score = 8.5`). Điểm số này mang tính chất nền tảng, phản ánh sở thích chung của người dùng dựa trên hành vi đám đông.

Bước 2: Tinh chỉnh bằng Content-Based qua thuật toán KNN

Để AI hiểu được gu nội dung thực sự, hệ thống quét lại lịch sử người dùng và áp dụng thuật toán KNN (K-Nearest Neighbors):

- Máy tính đối chiếu bộ phim ứng viên với lịch sử người dùng thông qua ma trận Cosine của BERT. Hệ thống sẽ lọc bỏ các phim không liên quan ($\text{Cosine} < \text{hoặc} = 0.1$) và chỉ giữ lại đúng 5 bộ phim giống nhất ($K=5$). Việc giới hạn $K=5$ giúp loại bỏ nhiễu, tránh tình trạng lấy trung bình của cả trăm bộ phim làm điểm số bị loãng.
- Sau đó, áp dụng công thức Trung bình trọng số: Phim nào trong lịch sử giống ứng viên hơn thì điểm số của phim đó sẽ có sức nặng cao hơn trong việc tính toán. Kết quả trả về là điểm nội dung (`cb_score`).

Bước 3: Lai ghép theo tỷ lệ trọng số

Bảng tinh chỉnh tham số:

Tỷ lệ lai ghép (SVD / BERT)	Hybrid RMSE
50% / 50%	1,2854

30% / 70%	1,3539
70% / 30%	1,2401
20% / 80%	1,396
80% / 20% (chọn)	1,2271

Điểm số cuối cùng được quyết định bằng cách cộng gộp điểm SVD và điểm Nội dung. Hệ thống áp dụng tỷ lệ trọng số lệch: **80% SVD - 20% BERT**.

$$\text{Hybrid_Score} = (\text{svd_score} * 0.8) + (\text{cb_score} * 0.2)$$

Lý do chọn tỷ lệ 80/20: SVD là thuật toán chuyên biệt về tối ưu hóa sai số (RMSE), do đó nó được giao 80% "quyền lực" để đảm bảo điểm dự đoán không bị sai lệch quá nhiều so với thực tế. BERT (Deep Learning) tuy hiểu nội dung cực tốt nhưng không giỏi dự đoán điểm số, do đó nó chỉ nắm 20% trọng số. Mức 20% này đóng vai trò như một lực kéo: Nếu bộ phim có cốt truyện cực kỳ hợp gu người xem, BERT sẽ kéo điểm tổng lên một chút để phim lọt vào danh sách Top 10, giúp hệ thống gợi ý đa dạng hơn và tìm ra những phim mà số đông chưa biết tới.

V. TRIỂN KHAI ỨNG DỤNG VÀ ĐÁNH GIÁ

1. Đánh giá độ chính xác :

Để đánh giá khách quan hiệu năng của hệ thống, đề tài tiến hành đo lường RMSE trên tập kiểm thử gồm 10.000 mẫu ngẫu nhiên. Bảng so sánh chỉ số RMSE giữa các mô hình

Thuật toán sử dụng	Tỷ lệ Trọng số	Chỉ số RMSE
Chỉ dùng SVD	100% SVD	1,2003

Hybrid: SVD + TF-IDF	50% SVD - 50% TF-IDF	1,2603
Hybrid: SVD + Sentence-BERT	80% SVD - 20% BERT	1,2271

Phân tích: Nhìn vào bảng kết quả, mô hình SVD độc lập cho ra chỉ số RMSE thấp nhất (tốt nhất). Khi lai ghép với AI phân tích nội dung, sai số có xu hướng tăng nhẹ. Hiện tượng này xuất phát từ bản chất toán học của các thuật toán:

- SVD là một thuật toán được sinh ra với mục tiêu tối thượng là "Giảm thiểu hàm mất mát RMSE". Do đó, nó cực kỳ xuất sắc trong việc dự đoán chính xác điểm số.
- Ngược lại, BERT hay TF-IDF là các thuật toán chuyên về đo lường sự tương đồng ngữ nghĩa. Khi chúng ta ép các mô hình này tham gia vào việc dự đoán một con số cụ thể (từ 1-10) thông qua trung bình trọng số (KNN), điểm số có xu hướng bị kéo về mức trung bình, làm loãng độ chính xác tuyệt đối của SVD.
- Thuật toán lai ghép với TF-IDF cho kết quả tệ nhất vì phương pháp đếm từ khóa tạo ra các liên kết lỏng lẻo, dẫn đến việc gợi ý những phim không thực sự liên quan.

Mặc dù SVD độc lập có RMSE tốt nhất, đề tài vẫn quyết định chọn Hybrid (SVD + BERT) với tỷ lệ trọng số 80/20 làm thuật toán cốt lõi. Sự đánh đổi một chút sai số RMSE này mang lại những lợi ích vô giá cho trải nghiệm người dùng:

1. **Giải quyết triệt để Cold-start:** SVD không thể gợi ý những bộ phim mới ra mắt chưa có ai chấm điểm. BERT giải quyết được điều này chỉ bằng cách đọc phần Tóm tắt nội dung của phim mới.
2. **Khai thác phim ẩn:** SVD có xu hướng thiên vị các bộ phim nổi tiếng. Nhờ 20% trọng số từ BERT, hệ thống có khả năng hiểu gu cốt truyện của người dùng, từ đó khai quật được những "viên ngọc ẩn" – những bộ phim ít người biết nhưng cực kỳ sát với sở thích cá nhân.

2. Triển khai giao diện :

2.1. Chức năng Đồng bộ MyAnimeList (MAL Sync)

Anime Recommender System

MyAnimeList SyncManual Import

TF-IDFBERT (Deep Learning)

Nhập Username MyAnimeList để hệ thống tự động đồng bộ lịch sử xem phim và tính toán gợi ý.
Trạng thái AI: Đang sử dụng thuật toán Học sâu (Sentence-BERT + SVD).

NHẬP USERNAME MYANIMELIST

trunghieu1504

Gợi ý ngay!

Đã đồng bộ 39 phim từ tài khoản **trunghieu1504**.

Top 10 Gợi ý

#1



Tsugu Tsugumomo
Thể loại: Action, Comedy, Supernatural, Ecchi, School, Seinen
Độ phù hợp: 11.32

#2



Fruits Basket 2nd Season
Thể loại: Slice of Life, Comedy, Supernatural, Drama, Romance, Shoujo
Độ phù hợp: 11.03

#3



InuYasha Movie 2: Kagami no Naka no Mugenjo
Thể loại: Action, Adventure, Comedy, Historical, Demons, Supernatural, Drama, Magic, Romance, Fantasy, Shounen
Độ phù hợp: 10.49

- Giải quyết bài toán nhập liệu cho người dùng cũ. Người dùng chỉ cần nhập Username, Frontend sẽ gọi API để Backend tiến hành cào toàn bộ lịch sử xem phim và điểm số từ nền tảng MyAnimeList.
- Chức năng này biến hệ thống thành một công cụ cá nhân hóa tự động hoàn toàn mà không đòi hỏi nỗ lực từ phía người dùng.

<https://myanimelist.net/animelist/trunghieu1504>

Organize, Discuss, Discover

MyAnimeList

HOW MUCH ANIME HAVE YOU SEEN?

All Anime Currently Watching Completed On Hold Dropped Plan to Watch 🔍

ALL ANIME							Stats	Filters
#	Image	Anime Title		Score	Type	Progress	Tags	
1		Baka & Test – Summon the Beasts 2	Add - More	8	TV	- / 13		
2		Battle Game in 5 Seconds 📌	Add - More	7	TV	- / 12		
3		O Maidens in Your Savage Season	Add - More	8	TV	- / 12		
4		5 Centimeters per Second	Add - More	6	Movie	3		
5		A Silent Voice	Add - More	9	Movie	1		

2.2. Chức năng Nhập liệu thủ công (Manual Import)

Anime Recommender System

MyAnimeList Sync

Manual Import

TF-IDF

BERT (Deep Learning)

Tự tìm kiếm phim, chấm điểm thủ công (1-10) để huấn luyện AI gợi ý theo sở thích cá nhân.

Trạng thái AI: Đang sử dụng thuật toán Học sâu (Sentence-BERT + SVD).

TÌM KIẾM ANIME ĐỂ THÊM VÀO DANH SÁCH

Nhập tên phim (VD: Naruto, Bleach...)

Danh sách chấm điểm (3)

Naruto

8 / 10

Xóa

Fairy Tail

9 / 10

Xóa

One Piece

8 / 10

Xóa

Tính toán AI với Danh sách này!

Top 10 Gợi ý dành riêng cho bạn

#1



One Piece Film: Z

Thể loại: Action, Adventure, Comedy, Drama, Fantasy, Shounen

Độ phù hợp: 7.69

#2



Magi: The Kingdom of Magic

Thể loại: Action, Adventure, Magic, Fantasy, Shounen

Độ phù hợp: 7.69

#3



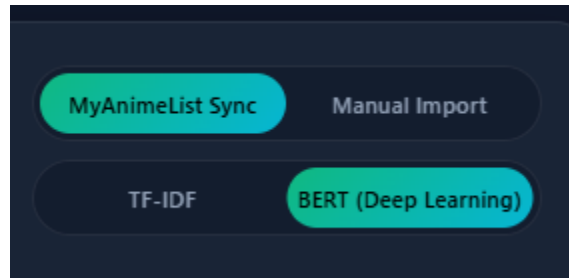
One Piece Film: Strong World

Thể loại: Action, Adventure, Comedy, Drama, Fantasy, Shounen

Độ phù hợp: 7.65

- Dành cho người dùng mới hoặc những người muốn thử nghiệm hệ thống. Hệ thống cung cấp một thanh tìm kiếm thông minh. Khi người dùng gõ từ khóa, API lập tức truy vấn trong 24.000 bộ phim để đưa ra gợi ý thả xuống.
- Phim sau khi chọn sẽ hiển thị dưới dạng danh sách với thanh trượt từ 1-10 để người dùng dễ dàng cung cấp dữ liệu ban đầu cho AI.

2.3. Chức năng AI Engine Toggle



- Ngay trên Header, hệ thống cung cấp một công tắc gạt cho phép người dùng chuyển đổi tức thì giữa thuật toán **TF-IDF (Classic)** và **BERT (Deep Learning)**.
- Khi chuyển đổi, cùng một danh sách đầu vào nhưng hệ thống sẽ tính toán và trả về 10 thẻ phim với sự khác biệt rõ rệt về chất lượng ngữ nghĩa, giúp người dùng trực tiếp cảm nhận được sự khác biệt của 2 thuật toán.

VI. KẾT LUẬN & TÀI LIỆU THAM KHẢO

1. Kết luận

Trải qua quá trình nghiên cứu, phân tích và triển khai, đề tài đã hoàn thành trọn vẹn các mục tiêu đặt ra ban đầu. Những kết quả nổi bật mà đề tài đạt được bao gồm:

- **Về mặt thuật toán:** Đã xây dựng thành công quy trình huấn luyện AI phức tạp trên môi trường Google Colab. Nắm vững và triển khai hiệu quả kỹ thuật tiền xử lý dữ liệu, đồng thời lai ghép thành công thuật toán Lọc cộng tác (SVD) với các mô hình phân tích ngôn ngữ tự nhiên (TF-IDF và Sentence-BERT).
- **Về mặt hệ thống:** Xây dựng thành công một ứng dụng Web hoàn chỉnh theo mô hình Client-Server. Đã đóng gói các mô hình AI thành công để giao tiếp mượt mà thông qua Flask RESTful API. Tích hợp các kỹ thuật xử lý đa luồng giúp tối ưu hóa thời gian phản hồi của máy chủ.
- **Về trải nghiệm người dùng (UX/UI):** Phát triển một giao diện người dùng hiện đại bằng ReactJS, cung cấp các tính năng tương tác thực tiễn như: Đồng bộ dữ liệu tự động từ MyAnimeList, tìm kiếm và chấm điểm thủ công (Manual Import), cùng công tắc chuyển đổi trực quan giữa các thuật toán AI.

2. Tài liệu tham khảo:

ThS. Bùi Thị Vân Anh (2023), *Báo cáo Seminar - Tìm hiểu về hệ thống khuyến nghị*

Vũ Hữu Tiệp (2017), *Machine Learning cơ bản: Content-based Recommendation Systems*. <https://machinelearningcoban.com/2017/05/17/contentbasedrecommendersys/>

<https://www.includehelp.com/ml-ai/root-mean-square%20error-rmse.aspx>